

Exercícios Propostos

Sistemas Distribuídos

Nome:

Exercício 1: Identificando Arquiteturas

1. WhatsApp

Cliente-Servidor. O sistema precisa de um ponto central para rotear mensagens, gerenciar presença e entregar mensagens a usuários offline. Escala horizontalmente com múltiplos servidores atrás de balanceadores. A tolerância a falhas vem de redundância nos servidores, não de distribuir confiança entre pares. A comunicação usa conexões persistentes entre cliente e servidor.

2. BitTorrent

Peer-to-Peer. O arquivo é dividido em pedaços e cada peer baixa e reenvia pedaços diferentes, o que distribui o custo de banda em vez de concentrar tudo num servidor. Isso dá escalabilidade proporcional ao número de peers e tolerância a falhas por replicação natural dos pedaços entre vários nós. A comunicação é direta entre peers, coordenada por um tracker ou por uma DHT.

3. Netflix

Microserviços. Catálogo, recomendação, cobrança e streaming são domínios independentes com necessidades de escala muito diferentes, então cada um escala separadamente. Uma falha na recomendação não derruba o streaming, o que isola falhas por serviço. A comunicação entre serviços é feita por APIs e por eventos assíncronos internamente.

4. Sistema bancário de caixa eletrônico (ATM)

Cliente-Servidor. Transações financeiras exigem consistência forte e um registro central único para evitar saque duplicado ou saldo divergente, o que um modelo P2P dificultaria. A escalabilidade vem de servidores centrais redundantes em data centers, e a tolerância a falhas de replicação síncrona do banco de dados central. A comunicação é do terminal ATM para o servidor central por um protocolo seguro.

5. Spotify

Microserviços. Busca, streaming, playlists, recomendação e anúncios são funções independentes mantidas por equipes separadas, cada uma escalando conforme sua própria demanda. Isso também isola falhas, já que a recomendação pode cair sem afetar a reprodução de música. A comunicação combina chamadas diretas entre serviços com um backbone de eventos para dados assíncronos, como histórico de audição.

Exercício 2: Detectando Problemas em Sistemas Distribuídos

1. Tipo de problema

Replicação inconsistente por falta de sincronização de estado. Como a requisição de curtir pode cair em qualquer um dos três servidores e cada um mantém sua própria contagem, não existe garantia de que os três convirjam para o mesmo valor antes de serem exibidos aos usuários.

2. Duas estratégias

Rotear sempre a mesma postagem para o mesmo servidor por hash consistente, combinado com um banco de dados de consistência forte para aquele contador, elimina a divergência na origem. Alternativamente, uma fila de eventos que serializa todas as curtidas de uma postagem antes de aplicá-las às réplicas garante que todos os servidores apliquem as mudanças na mesma ordem.

Exercício 3: Escolhendo Comunicação entre Serviços

Resposta

B, comunicação via fila de mensagens (Kafka ou RabbitMQ).

Justificativa

Em acoplamento, o serviço de pedidos publica o evento de pedido criado sem precisar saber se pagamento e envio estão disponíveis no momento, o que os desacopla. Em escalabilidade, cada serviço consome da fila no seu próprio ritmo, então um pico de pedidos não trava o pagamento nem o envio. Em tolerância a falhas, se o serviço de pagamento cair, as mensagens continuam na fila e são processadas quando ele voltar, em vez de o pedido inteiro falhar como aconteceria numa chamada síncrona via REST. Esse desenho é o padrão Saga para transações distribuídas.

Exercício 4: Problemas de Churn em P2P

1. Problemas causados

A descoberta de recursos fica mais lenta porque um peer pode sair antes de responder a uma busca em andamento. Conexões ativas caem e exigem reconexão e retransmissão do que já tinha sido baixado. A topologia muda constantemente, o que deixa tabelas de roteamento desatualizadas e aumenta a instabilidade da rede como um todo.

2. Duas soluções

Uma DHT redistribui automaticamente a responsabilidade por cada chave entre os nós restantes quando alguém entra ou sai, mantendo a busca funcional sem coordenação central. Replicar cada arquivo ou pedaço de arquivo em vários peers garante que a saída de um nó não torne o conteúdo indisponível.

Exercício 5: Projetando um Sistema Distribuído

1. Tipo de arquitetura

Event-driven combinada com microsserviços. Cada atualização de localização de um entregador é um evento publicado num barramento de eventos, e serviços independentes consomem esses eventos para fins diferentes, como cálculo de rota, estimativa de chegada e notificação ao aplicativo.

2. Mecanismo de comunicação

Publicação e assinatura por um broker de mensagens, como Kafka, entre os serviços de backend, e WebSocket entre o backend e o aplicativo do cliente para empurrar a posição em tempo real sem que o app precise ficar consultando o servidor.

3. Estratégia de escalabilidade

Particionar os tópicos do broker por região geográfica ou por identificador do entregador permite processar atualizações em paralelo, e os consumidores escalam horizontalmente conforme o volume de eventos aumenta.

4. Tratamento de falhas de servidores

Réplicas dos brokers de mensagens evitam perda de eventos se um nó cair. Consumidores organizados em grupo se reequilibram automaticamente quando uma instância falha, e a posição mais recente de cada entregador fica num banco distribuído com replicação, como Cassandra, para não perder dados recentes na falha de um único nó.